

Online Complaint Registration System (MERN Stack)

Full Stack Development Project Documentation

Date: 7 August 2026 | Project Stack: MongoDB, Express.js, React.js, Node.js

1. INTRODUCTION

Project Title: Online Complaint Registration System (MERN Stack)

This document provides complete architectural details, API specifications, setup guidelines, and functional overviews for the Online Complaint Registration web application platform.

2. PROJECT OVERVIEW

Purpose:

The Online Complaint Registration System streamlines the process of connecting citizens with civic/government departments. It enables citizens to file, track, and manage grievances through an intuitive interface, while giving officers and admins simple, secure tools to manage assignment, resolution, and platform activity.

Features:

- Role-based registration and login for Citizens, Officers, and Admins (JWT authentication)
- Complaint filing with category, sub-category, location, and evidence attachments
- Real-time complaint status tracking, escalation, and reassignment
- Secure upload and storage of photo/video evidence and supporting documents
- Admin dashboard for department verification, workload balancing, and analytics
- Email/SMS notifications for filing confirmation, status changes, and resolution

3. ARCHITECTURE

Frontend (React):

Built with React.js using a component-based structure. React Router handles client-side navigation between pages (login, complaint filing, tracking, dashboards). Axios manages HTTP communication with the backend REST API, and Bootstrap provides a responsive UI. Components are organized by role (Citizen, Officer, Admin) with shared components for common elements like navigation bars and forms.

Backend (Node.js & Express.js):

The backend follows the MVC (Model-View-Controller) pattern. Express.js handles routing and middleware (JSON parsing, CORS, JWT authentication). Controllers contain the business logic for each domain (Auth, Complaint, Department, Document, Admin), keeping routes thin and logic centralized and testable.

Database (MongoDB):

MongoDB stores three core collections via Mongoose schemas:

- Users: name, email, hashed password, role (citizen/officer/admin), profile info
- Departments: name, category mapping, assigned officers, service-level agreement (SLA)
- Complaints: reference to Citizen ID, category, description, location, status (filed/in-review/assigned/resolved/closed), uploaded evidence references

4. SETUP INSTRUCTIONS

Prerequisites:

- Node.js (v16 or higher) and npm
- MongoDB (local installation or a MongoDB Atlas cluster)
- Git

Installation:

```
git clone <repository-url>
cd online-complaint-registration
Install backend dependencies:
cd server
npm install
Install frontend dependencies:
cd ../client
npm install
Create a .env file inside the server folder with:
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret_key
PORT=5000
```

5. FOLDER STRUCTURE

Client (React frontend):

```
client/src/
  components/ (navbar, forms, cards)
  pages/ (Login, Register, FileComplaint, TrackComplaint, Dashboard)
  services/ (axios API call functions)
  context/ (auth context for logged-in user & token)
  App.js, index.js
```

Server (Node.js backend):

```
server/
  models/ (User.js, Department.js, Complaint.js)
  routes/ (authRoutes.js, complaintRoutes.js, departmentRoutes.js, adminRoutes.js)
  controllers/ (authController.js, complaintController.js, departmentController.js,
  adminController.js)
  middleware/ (authMiddleware.js JWT verification & role checks)
  config/ (db.js MongoDB connection)
  server.js
```

6. RUNNING THE APPLICATION

Backend:

```
cd server
npm start
```

Frontend:

```
cd client
npm start
```

The frontend runs on <http://localhost:3000> and the backend API runs on <http://localhost:5000> by default.

7. API DOCUMENTATION

Method	Endpoint	Description	Request Body / Params
POST	/api/auth/register	Register a new user (citizen/officer)	{ name, email, password, role }
POST	/api/auth/login	Authenticate user and return JWT token	{ email, password }
POST	/api/complaints	File a new complaint (Citizen only)	{ category, description, location, attachments }
GET	/api/complaints	Get list of complaints (supports category/status/location filters)	Query: ?category=&status=&location=
GET	/api/complaints/:id	Get a single complaint's full details	Param: complaint id

PUT	/api/complaints/:id/status	Update complaint status (Officer/Admin only)	{ status, remarks }
GET	/api/complaints/user/:userId	Get all complaints filed by a citizen	Param: user id
POST	/api/documents/upload	Upload evidence for a complaint	multipart/form-data: file, complaintId
PUT	/api/admin/departments/:id/assign	Assign complaint to a department/officer (Admin only)	{ complaintId, officerId }

Example response (POST /api/auth/login):

```
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": "64f...",
    "name": "Jane Doe",
    "email": "jane@example.com",
    "role": "citizen"
  }
}
```

8. AUTHENTICATION

Authentication is handled using JSON Web Tokens (JWT). On successful login, the server signs a token containing the user's ID and role, which the client stores (e.g., in memory / React context) and attaches to the Authorization header (Bearer token) on subsequent requests.

Passwords are hashed with bcrypt before being stored - plain-text passwords are never saved.

An authentication middleware verifies the JWT on protected routes and rejects invalid/expired tokens.

A role-based authorization check restricts endpoints (e.g., only Admins can assign departments, only Officers can update status on their own queue).

9. USER INTERFACE

[Insert screenshots or GIFs here showcasing the Login/Register page, File Complaint page, Track Complaint page, Citizen Dashboard, Officer Dashboard, and Admin Dashboard.]

10. TESTING

API endpoints were tested using Postman to verify request/response handling, status codes, and authentication behavior.

Manual User Acceptance Testing (UAT) was performed to validate end-to-end flows such as registration, complaint filing, status tracking, escalation, and evidence upload (see the separate UAT document for detailed test cases and results).

11. SCREENSHOTS OR DEMO

[Add screenshots of key screens here, or a link to a demo video / hosted deployment showcasing the working application, as required by your internship deliverables.]

12. KNOWN ISSUES

- Complaint status list does not refresh in real time after an officer updates a status (requires manual refresh).
- File size limit for evidence uploads is not validated on the frontend, which can cause a server-side error for large files.
- Admin's department-assignment button remains clickable briefly after submission, which could allow a duplicate assignment click.

13. FUTURE ENHANCEMENTS

- AI-based auto-categorization and priority scoring of complaints
- Online payment gateway for applicable service/penalty fees
- Citizen ratings and feedback analytics for department performance
- In-app real-time chat between citizen and assigned officer

- Multi-language support for wider accessibility
- Push notifications and map-based hotspot visualization for admins